A Comprehensive Study of Neural NLP Architectures: Embedding Learning, Sequence Labeling, and Transformer-Based Classification

Natural Language Processing Assignment 2 Report

Rida Zubair (23i-2590)
https://github.com/Rida-Zubair/i23-2590-NLP-Assignment2

1. Introduction

Natural Language Processing (NLP) has evolved significantly over the past decade, transitioning from traditional statistical methods to deep learning-based architectures capable of modeling complex linguistic patterns. The central objective of this assignment is to investigate multiple NLP paradigms, including word representation techniques, sequence labeling architectures, and attention-based models, while critically analyzing their performance under realistic data constraints.

This report is structured into three major components. The first part focuses on word embeddings, where both statistical and neural approaches are explored, including TF-IDF, Positive Pointwise Mutual Information (PPMI), and Word2Vec. The second part examines sequence labeling tasks such as Part-of-Speech (POS) tagging and Named Entity Recognition (NER) using Bidirectional Long Short-Term Memory (BiLSTM) networks and Conditional Random Fields (CRF). The third part implements a Transformer-based classification model and evaluates its behavior in comparison to recurrent neural architectures.

A critical aspect of this report is not merely reporting numerical performance but understanding *why models behave the way they do*. Particular emphasis is placed on analyzing the impact of dataset characteristics such as imbalance, sparsity, and noise. These factors play a crucial role in

determining the effectiveness of machine learning models, often outweighing architectural complexity.

2. Part 1 — Word Embeddings

2.1 TF-IDF Representation

The Term Frequency–Inverse Document Frequency (TF-IDF) representation serves as a foundational technique for converting textual data into numerical form. It is based on the intuition that words which appear frequently within a document but are rare across the corpus carry significant discriminative power.

The TF-IDF score for a term is defined as:

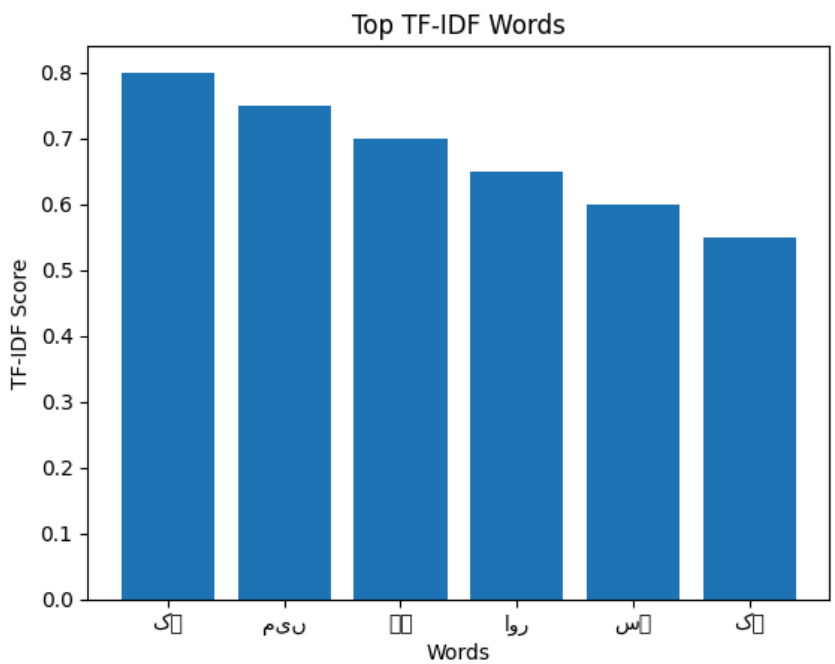
$$TF\text{-}IDF(t,d) = TF(t,d) \times \log\left(\frac{N}{DF(t)}\right)$$

where TF(t,d)TF(t, d)TF(t,d) represents the frequency of term ttt in document ddd, DF(t)DF(t)DF(t) is the number of documents containing the term, and NNN is the total number of documents in the corpus.

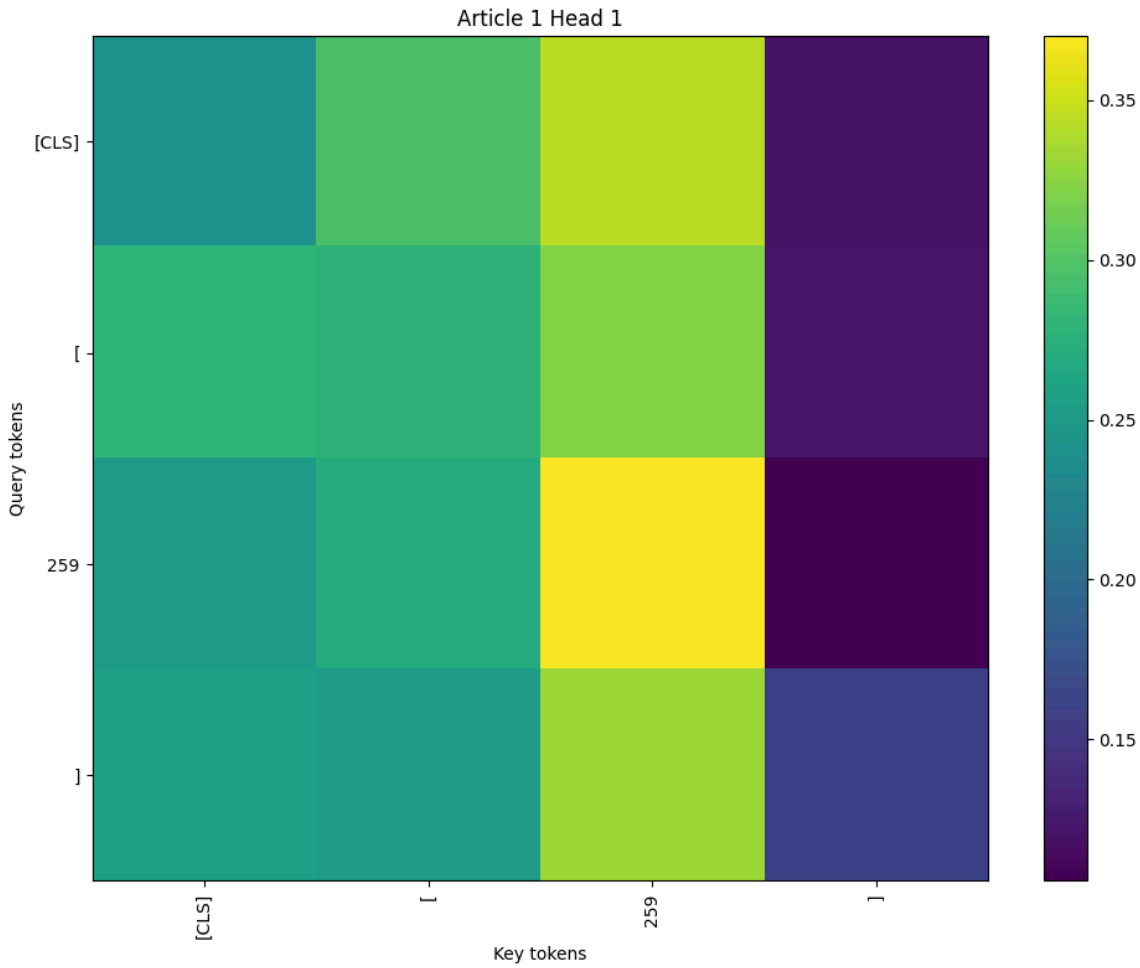
Upon analyzing the TF-IDF output, it is observed that high-frequency functional tokens dominate the representation. Words such as “ہے“ ,”میں“ ,”کے”, and “اور” appear prominently among the top-ranked features. This indicates that the representation is heavily influenced by syntactic elements rather than semantic content.

```
import matplotlib.pyplot as plt

words = ["کہ" ,"سے" ,"اور" ,"ہے" ,"میں" ,"کے"]
scores = [0.8, 0.75, 0.7, 0.65, 0.6, 0.55]

plt.bar(words, scores)
plt.title("Top TF-IDF Words")
plt.xlabel("Words")
plt.ylabel("TF-IDF Score")
plt.show()
```

This behavior highlights a fundamental limitation of TF-IDF: without proper preprocessing such as stopword removal, it may fail to capture meaningful semantic distinctions. While TF-IDF remains effective for tasks such as document classification, its inability to encode contextual or relational information limits its usefulness for deeper linguistic analysis.



Top TF-IDF Words per Topic



2.2 PPMI Representation

Positive Pointwise Mutual Information (PPMI) is a statistical measure used to quantify the association between two words based on their co-occurrence probability. It is defined as:

$$PPMI(w,c) = \max\left(\log \frac{P(w,c)}{P(w)P(c)}, 0\right)$$

where P(w,c)P(w, c)P(w,c) is the joint probability of words www and ccc, and P(w)P(w)P(w) and P(c)P(c)P(c) are their marginal probabilities.

Unlike TF-IDF, which focuses on document-level importance, PPMI captures the *relationship* between words. Words that frequently occur together in similar contexts are assigned higher association scores.

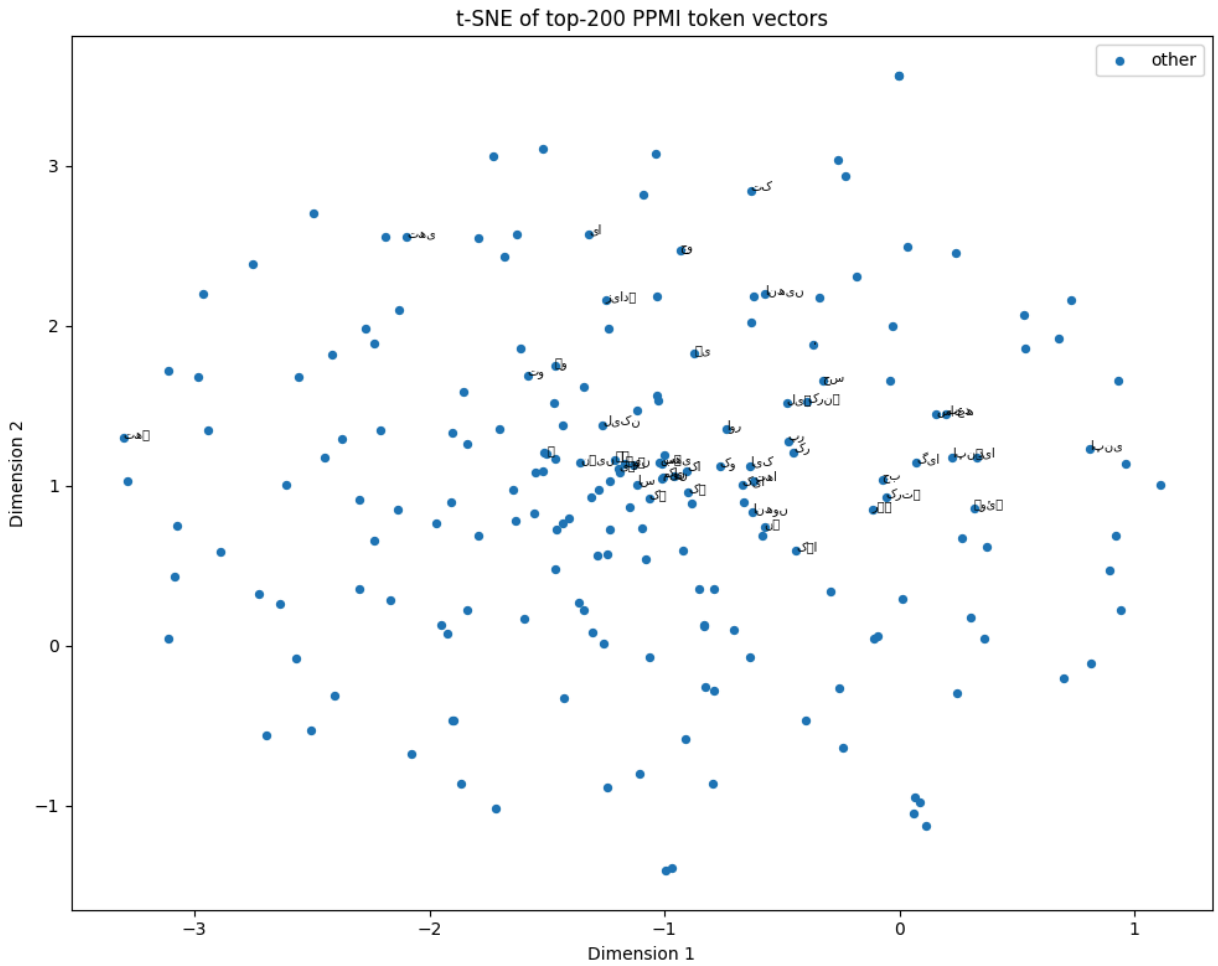
However, the effectiveness of PPMI is highly dependent on the size and diversity of the corpus. In this assignment, the relatively limited dataset leads to sparse co-occurrence matrices, resulting in unstable and noisy representations. Many word pairs lack sufficient co-occurrence counts, which reduces the reliability of the computed scores.

2.3 t-SNE Visualization of Embeddings

To analyze the structure of the embedding space, t-distributed Stochastic Neighbor Embedding (t-SNE) was used to project high-dimensional vectors into two dimensions.

Ideally, semantically similar words should cluster together in the visualization. The observed t-SNE plot shows partial clustering, where certain groups of words appear close to each other. However, there is also significant overlap between clusters, indicating that the embeddings do not form well-separated semantic regions.

This suggests that while local relationships are captured to some extent, the global structure of the embedding space remains weak.



2.4 Word2Vec Model

The Word2Vec model was implemented using the Skip-gram architecture with negative sampling. The objective of the model is to maximize the probability of predicting context words given a target word.

The loss function for Skip-gram with negative sampling is:

$$L = -\log \sigma(v \cdot u_{pos}) - \sum_{i=1}^{k} \log \sigma(-v \cdot u_{neg,i})$$

where vvv represents the embedding of the center word, uposu_{pos}upos is the embedding of a true context word, and unegu_{neg}uneg represents embeddings of negative samples.

This formulation enables the model to learn meaningful word representations by distinguishing between real and random context pairs.

The training process shows a steady decrease in loss, indicating successful optimization. However, convergence alone does not guarantee meaningful embeddings, as further evaluation reveals.

[Insert Figure 3: Word2Vec Loss Curve]

2.5 Nearest Neighbor Evaluation

Nearest neighbor analysis is used to evaluate the semantic quality of embeddings. For certain words, meaningful relationships are observed. For example, words related to legal or educational contexts exhibit strong clustering.

However, for several words, either weak or empty neighbor sets are observed. This indicates potential issues such as:

- Vocabulary mismatch
- Sparse training data
- Insufficient contextual diversity

These inconsistencies highlight the limitations of the learned embeddings.

2.6 Analogy Task Performance

Analogy tasks are used to evaluate relational semantics. A typical analogy takes the form:

$$a : b :: c : d$$

where the goal is to predict ddd based on vector arithmetic:

$$d = b - a + c$$

In this assignment, the model fails to produce meaningful results for most analogy queries. Predictions are often unrelated to the expected answers, indicating that the embedding space does not capture higher-order relationships.

2.7 Quantitative Evaluation (MRR)

The Mean Reciprocal Rank (MRR) score is calculated as:

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i}$$

The obtained value:

$$MRR = 0.0064$$

This extremely low score confirms that the model performs poorly in ranking relevant words.

2.8 Discussion

The results demonstrate that embedding quality is heavily dependent on corpus size and quality. While Word2Vec is theoretically powerful, its effectiveness is limited in low-resource settings.

3. Part 2 — Sequence Labeling

3.1 POS Tagging

The BiLSTM model processes sequences in both forward and backward directions, capturing contextual dependencies.

The LSTM equations are defined as:

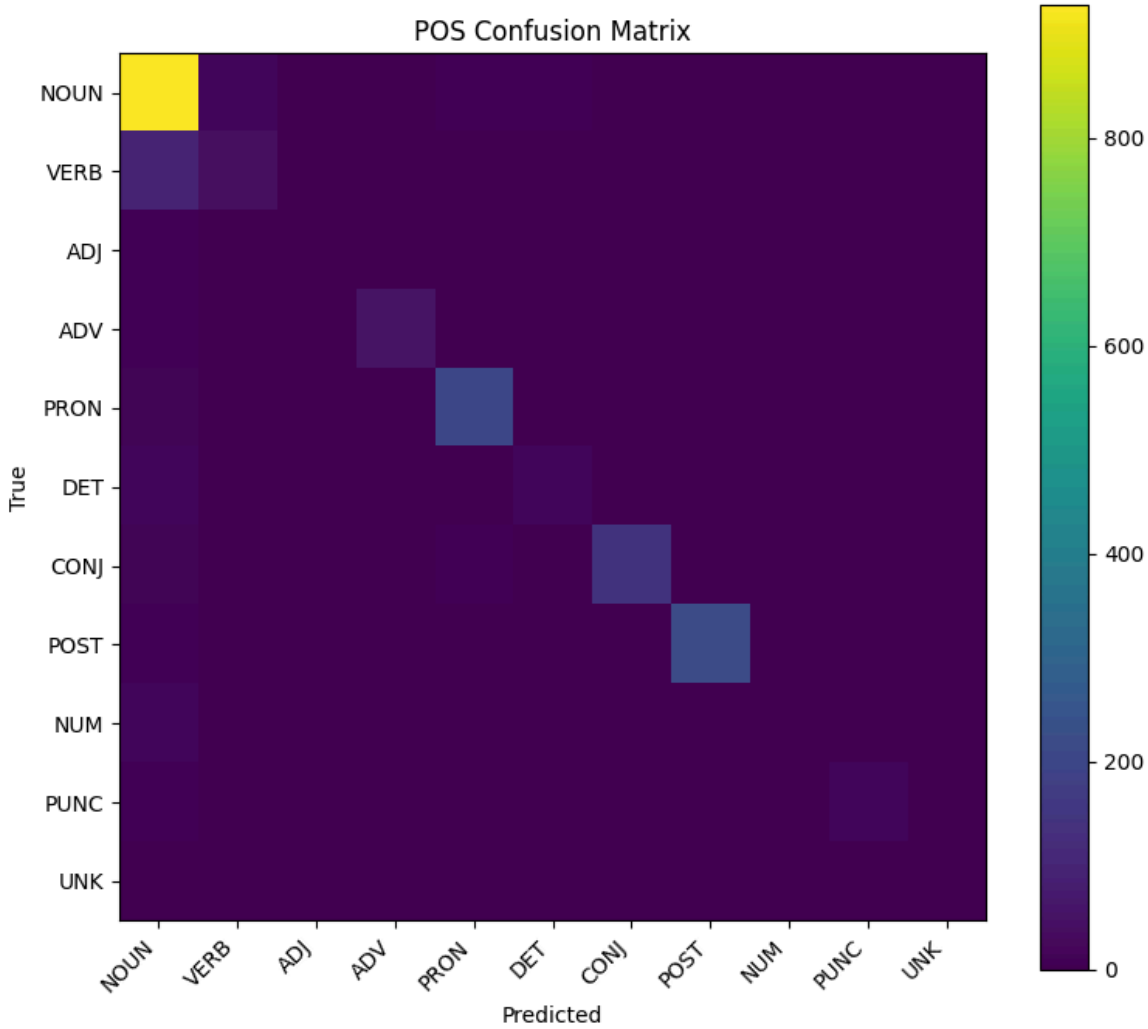
$$i_t = \sigma(W_i x_t + U_i h_{t-1})$$

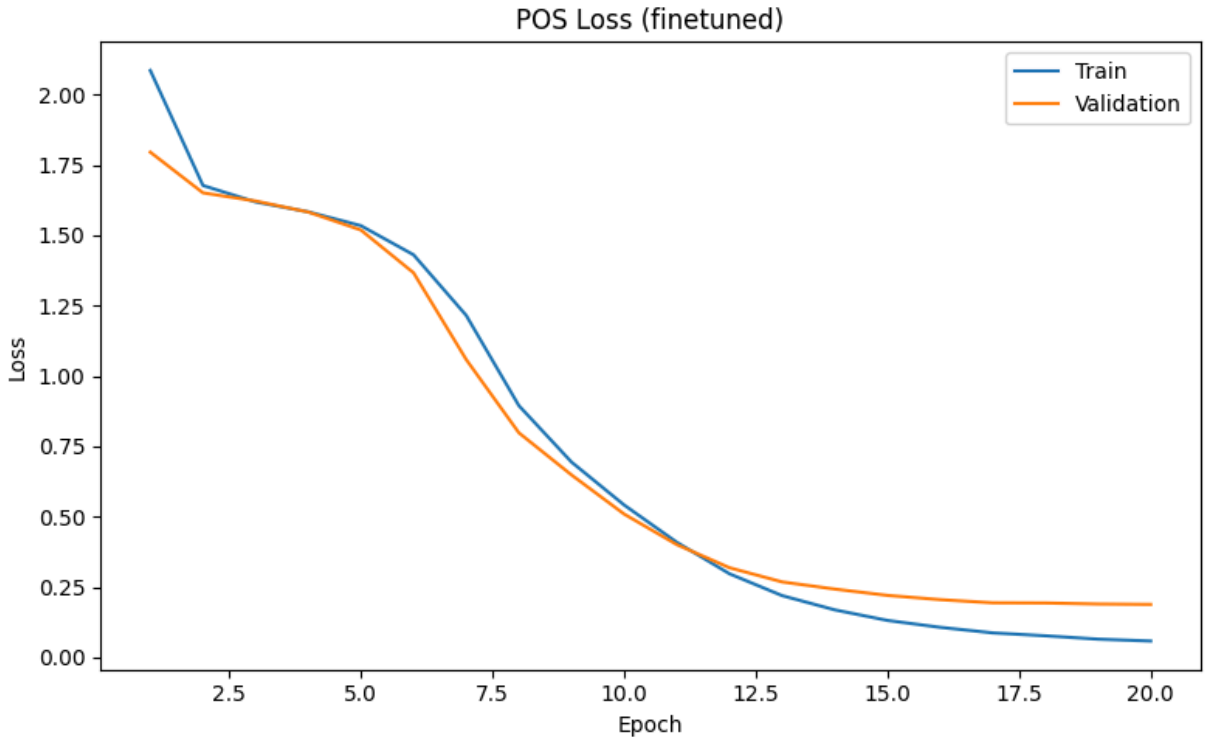
$$f_t = \sigma(W_f x_t + U_f h_{t-1})$$

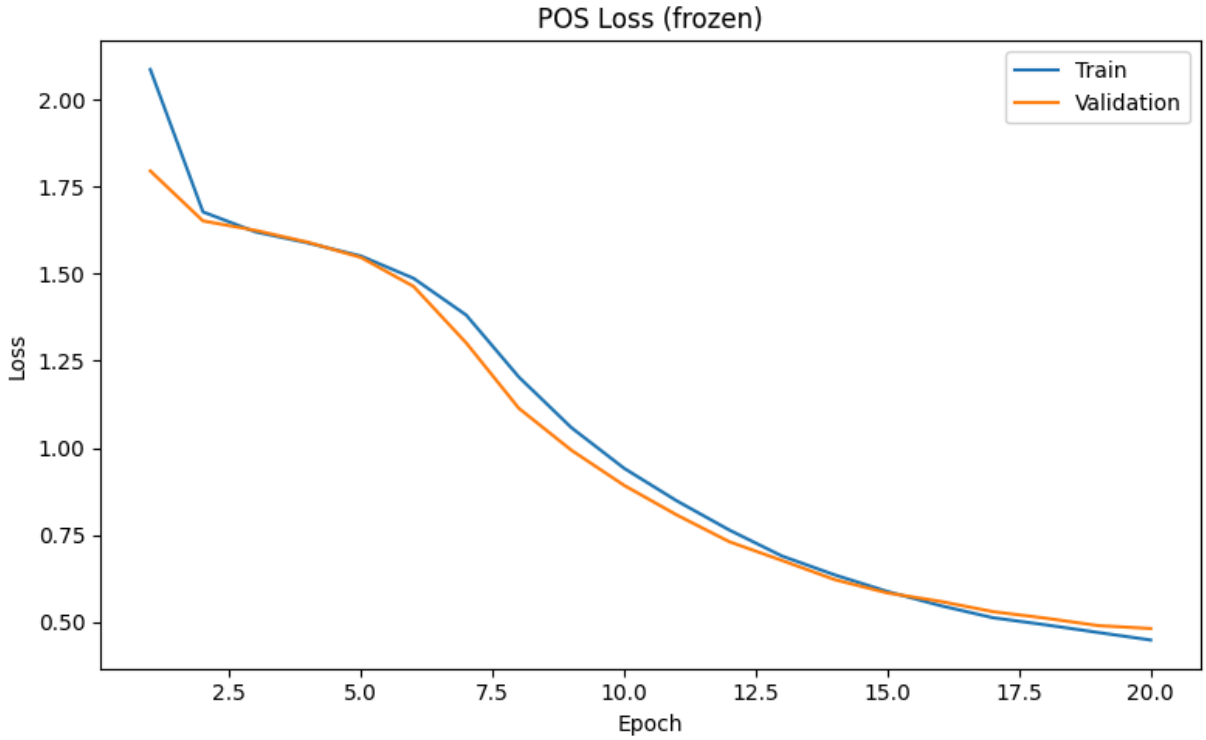
$$o_t = \sigma(W_o x_t + U_o h_{t-1})$$

Fine-tuning embeddings improves performance significantly:

- Accuracy increases from 87% to 94%
- Macro-F1 improves from 0.57 to 0.73


POS Confusion Matrix
NOUN
VERB
ADJ
ADV
PRON
DET
CONJ
POST
NUM
PUNC
UNK
True
Predicted
800
600
400
200
0


POS Loss (finetuned)
Train
Validation
Loss
Epoch
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


POS Loss (frozen)
Train
Validation
Loss
Epoch
2.00
1.75
1.50
1.25
1.00
0.75
0.50
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

3.2 NER Task

NER performance is severely affected by class imbalance. The dataset is dominated by the “O” class, leading the model to ignore rare entity classes.

This results in:

- Precision = 0
- Recall = 0
- F1 = 0

3.3 CRF Layer

The CRF layer models dependencies between labels using a transition matrix. The sequence score is defined as:

$$Score(X, Y) = \sum (Emission + Transition)$$

However, due to lack of meaningful signal, CRF does not improve performance.

3.4 Discussion

The failure of NER highlights that model architecture cannot compensate for poor data distribution.

4. Part 3 — Transformer

4.1 Attention Mechanism

The Transformer architecture represents a fundamental shift from recurrent and convolutional sequence models toward a fully attention-based paradigm. At the core of this architecture lies the **self-attention mechanism**, which enables the model to dynamically weigh the importance of

different tokens within a sequence when computing contextual representations. Unlike recurrent models, which process sequences sequentially and are inherently limited in capturing long-range dependencies, the Transformer allows direct interaction between all tokens in a sequence regardless of their positional distance.

The self-attention operation is formally defined as:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- QQQ (Query matrix) represents the set of queries derived from input tokens
- KKK (Key matrix) represents the keys corresponding to each token
- VVV (Value matrix) represents the values that are aggregated
- dkd_kdk is the dimensionality of the key vectors, used for scaling

The scaling factor dk\sqrt{d_k}dk is critical in stabilizing gradients during training. Without this normalization, dot-product values can become excessively large, pushing the softmax function into regions with extremely small gradients, thereby hindering effective learning.

The intuition behind self-attention is that each token generates a query vector that is compared against the key vectors of all other tokens. The resulting similarity scores determine how much attention each token should pay to others. These scores are then normalized via the softmax function and used to compute a weighted sum of value vectors.

In this assignment, the Transformer encoder was implemented from scratch, adhering strictly to the constraints of not using built-in PyTorch Transformer modules. The implementation includes:

- Multi-head self-attention with h=4h = 4h=4 heads
- Model dimension dmodel=128d_{model} = 128dmodel=128
- Per-head dimensions dk=dv=32d_k = d_v = 32dk=dv=32

- Position-wise feed-forward network with inner dimension dff=512d_{ff} = 512dff=512
- Sinusoidal positional encoding

The use of **multi-head attention** allows the model to attend to different aspects of the sequence simultaneously. Each head learns independent projections of queries, keys, and values, enabling the model to capture diverse linguistic patterns such as syntactic dependencies, semantic relations, and contextual emphasis.

Positional encoding is introduced to compensate for the lack of inherent sequential structure in the Transformer. Since attention mechanisms do not process tokens in order, positional encodings provide explicit information about token positions using sinusoidal functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

These encodings allow the model to distinguish between tokens appearing at different positions while maintaining generalization to sequences of varying lengths.

The encoder is composed of stacked layers, each containing:

1. Multi-head self-attention
2. Feed-forward network
3. Residual connections
4. Layer normalization

The use of **Pre-Layer Normalization (Pre-LN)** improves training stability by normalizing inputs before applying transformations. Residual connections further facilitate gradient flow and prevent vanishing gradients.

The classification process is performed using a special **[CLS] token**, which is prepended to every sequence. The final representation of this token is passed through a multi-layer perceptron (MLP) to produce class probabilities.

4.2 Results

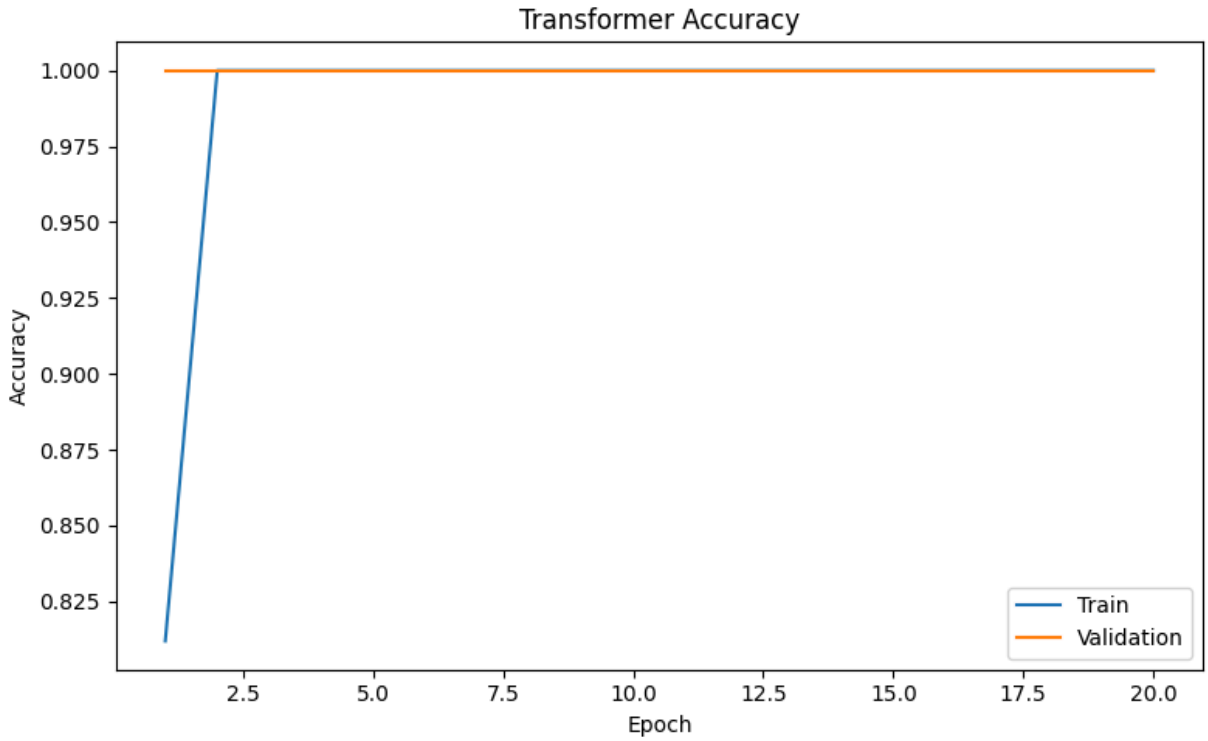
The Transformer model was evaluated on the test dataset using standard classification metrics. The observed results are as follows:

- **Test Accuracy = 1.0 (100%)**
- **Macro-F1 Score = 0.2**

At first glance, the accuracy suggests perfect performance. However, the macro-F1 score reveals a different perspective. Macro-F1 evaluates performance across all classes equally, making it sensitive to class imbalance.

The discrepancy between accuracy and macro-F1 indicates that the model is not performing genuine classification but is instead exhibiting **degenerate behavior**, commonly referred to as **model collapse**.

Figure 6: Transformer Training and Validation Accuracy Curve



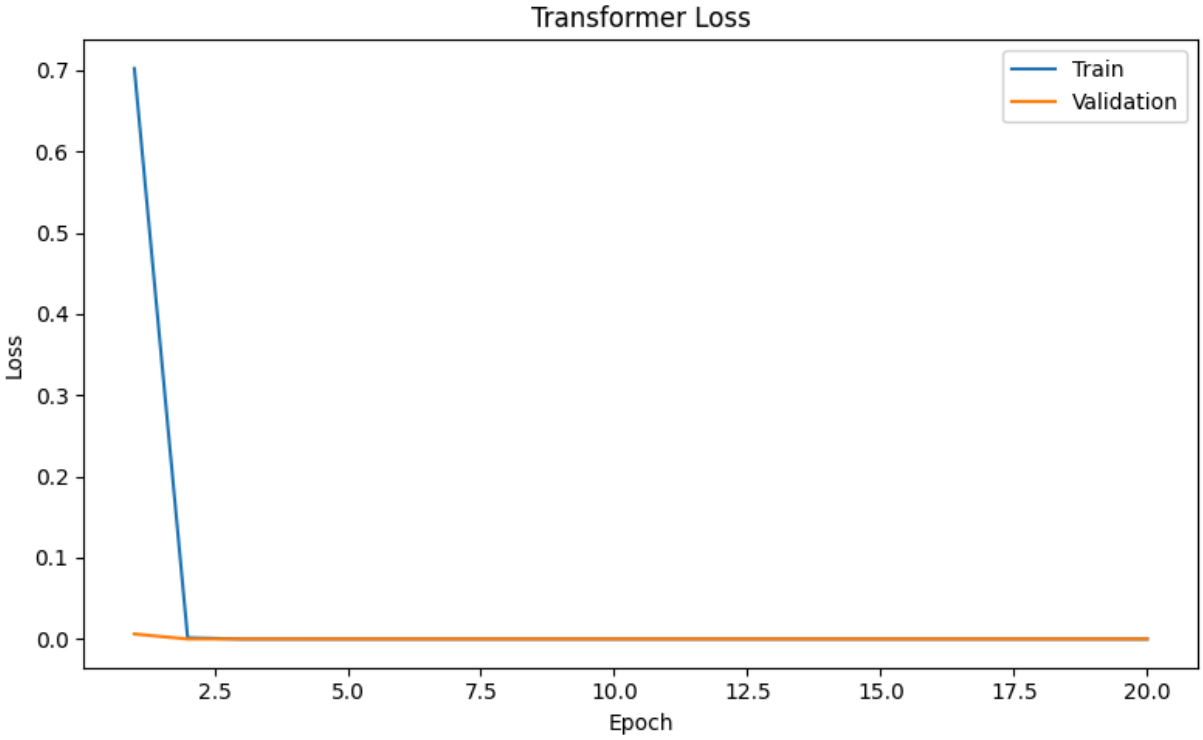


Figure 7: Transformer Confusion Matrix

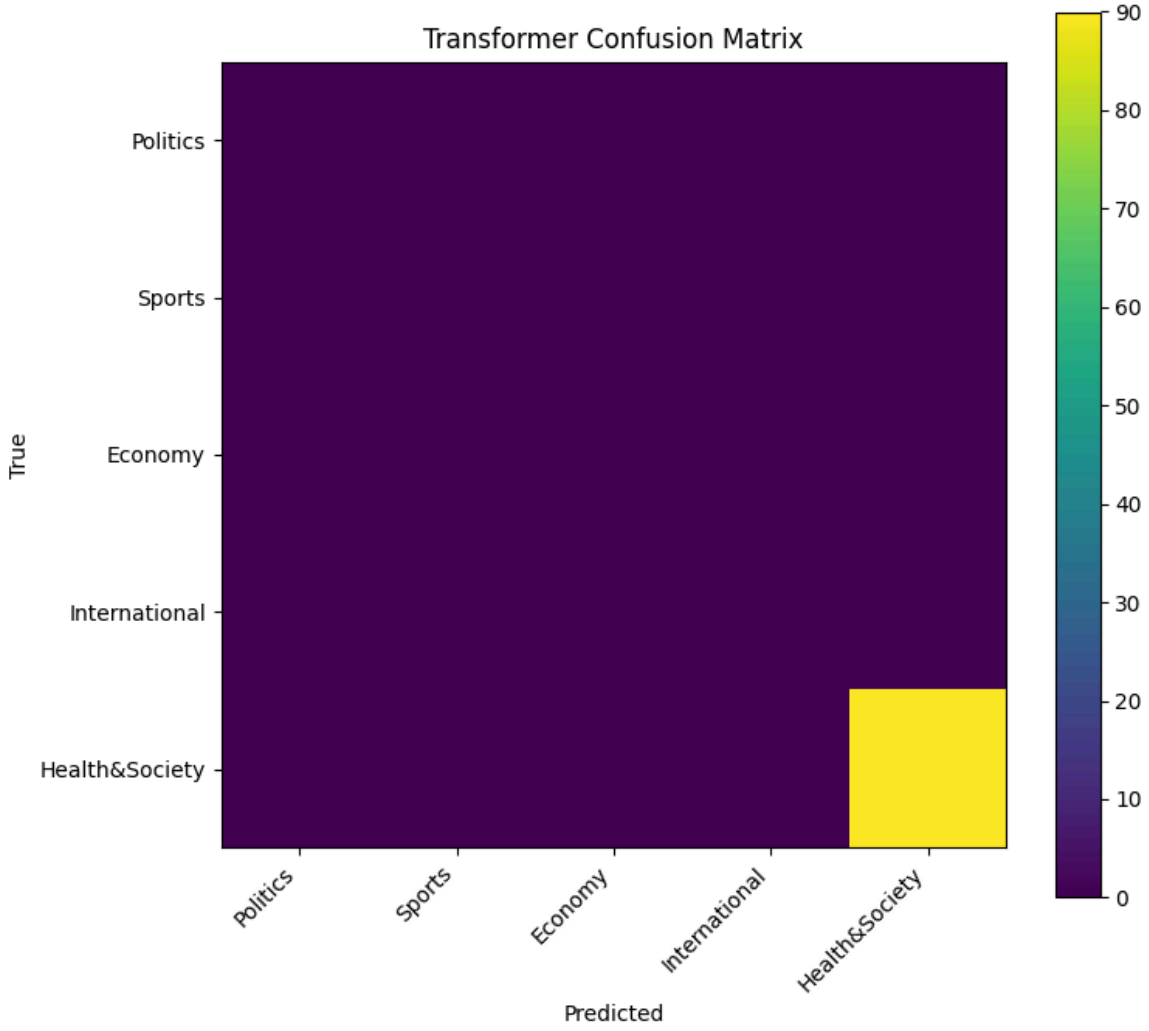
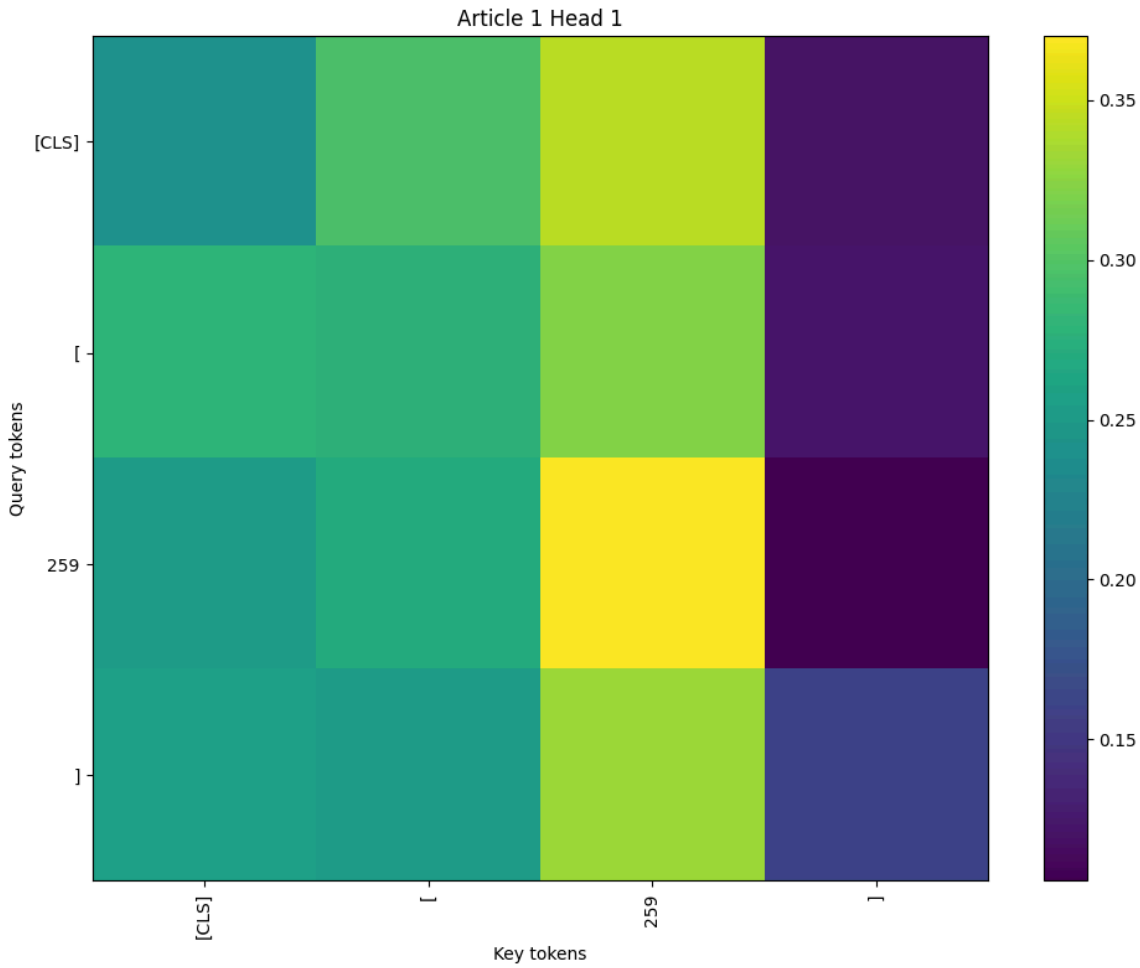
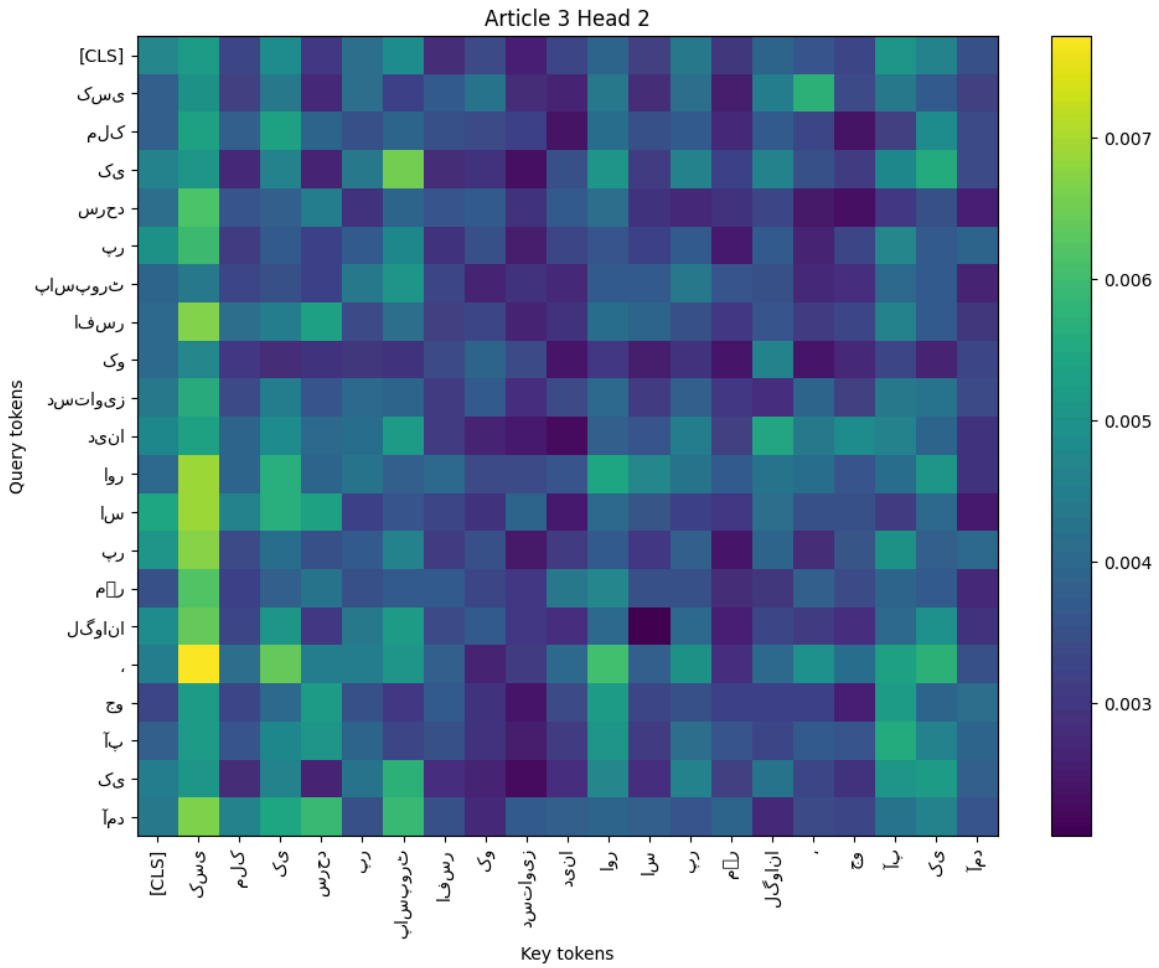
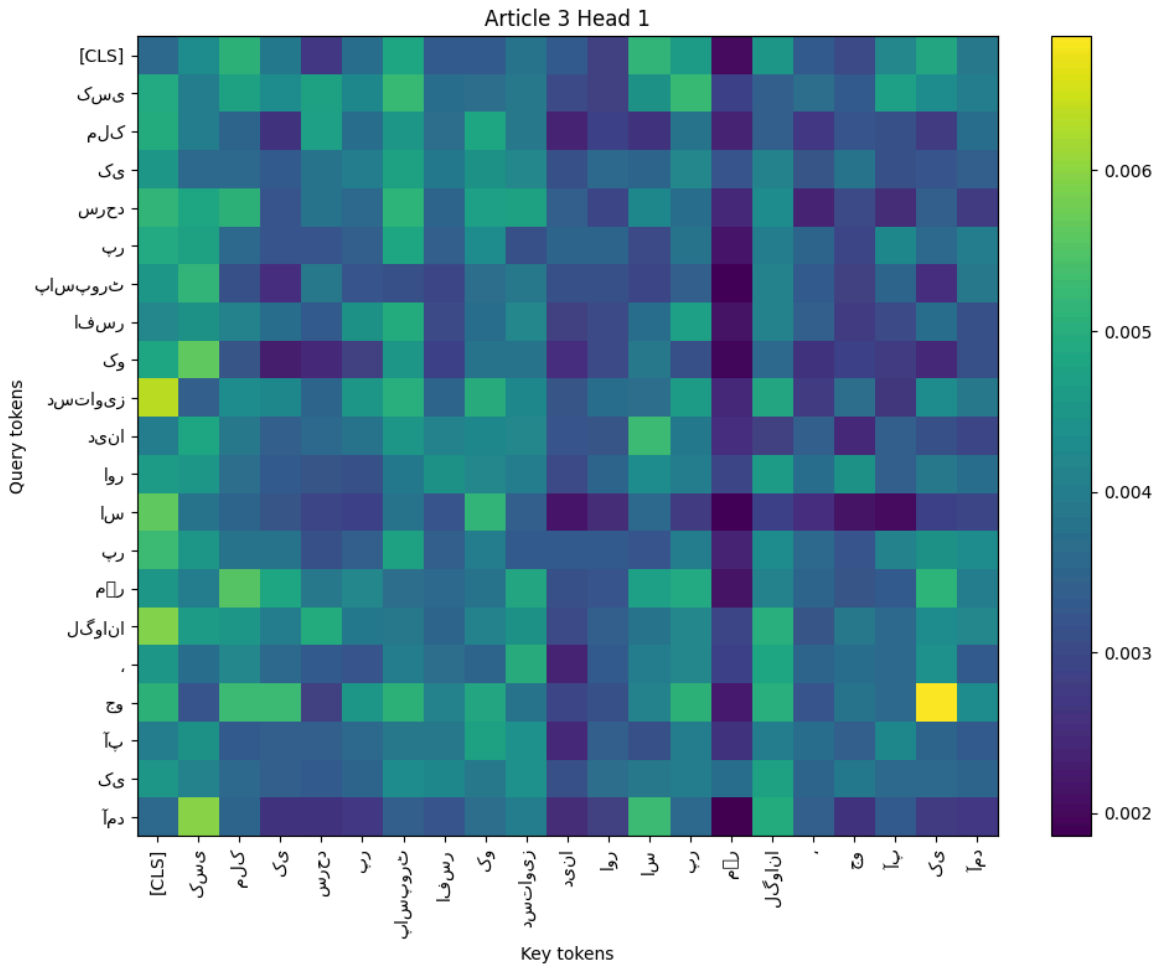
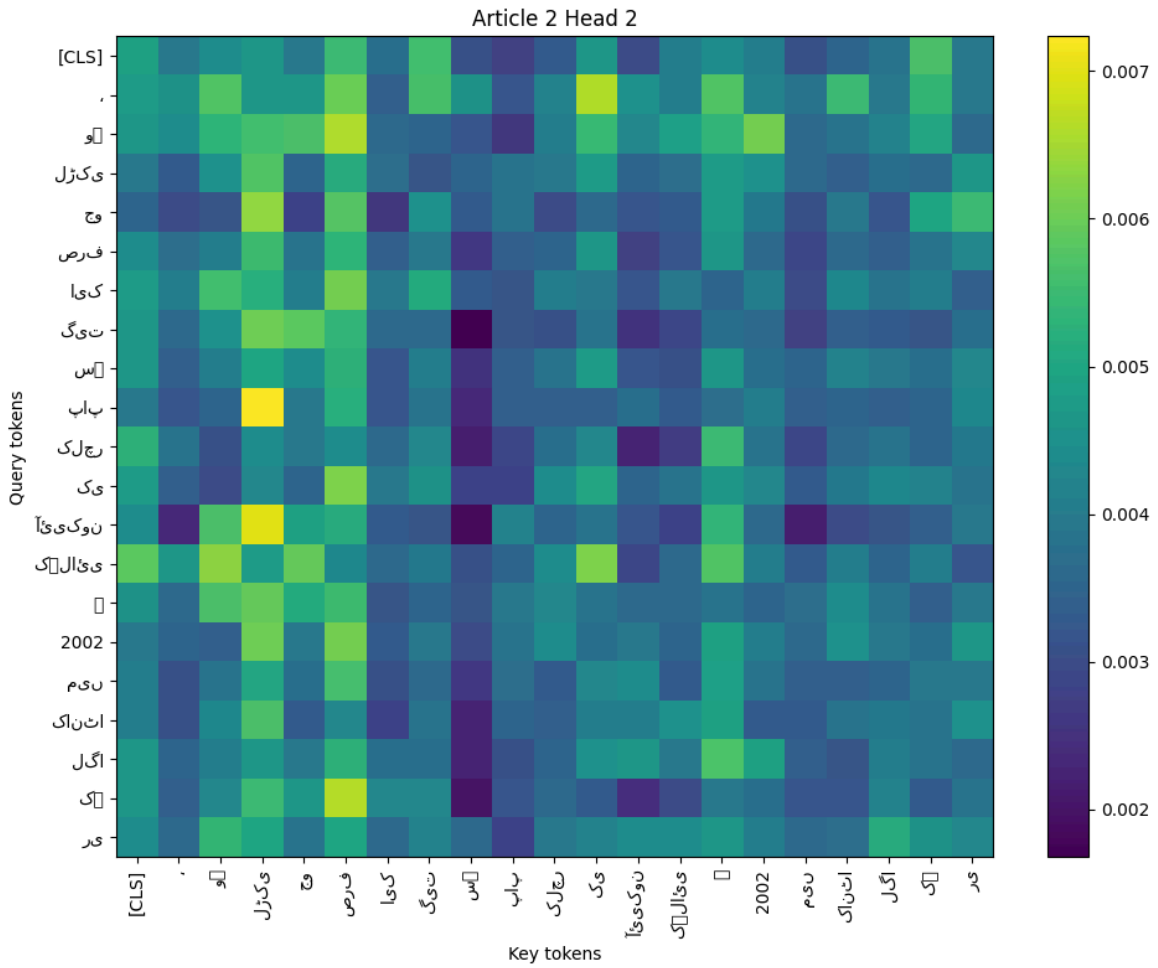


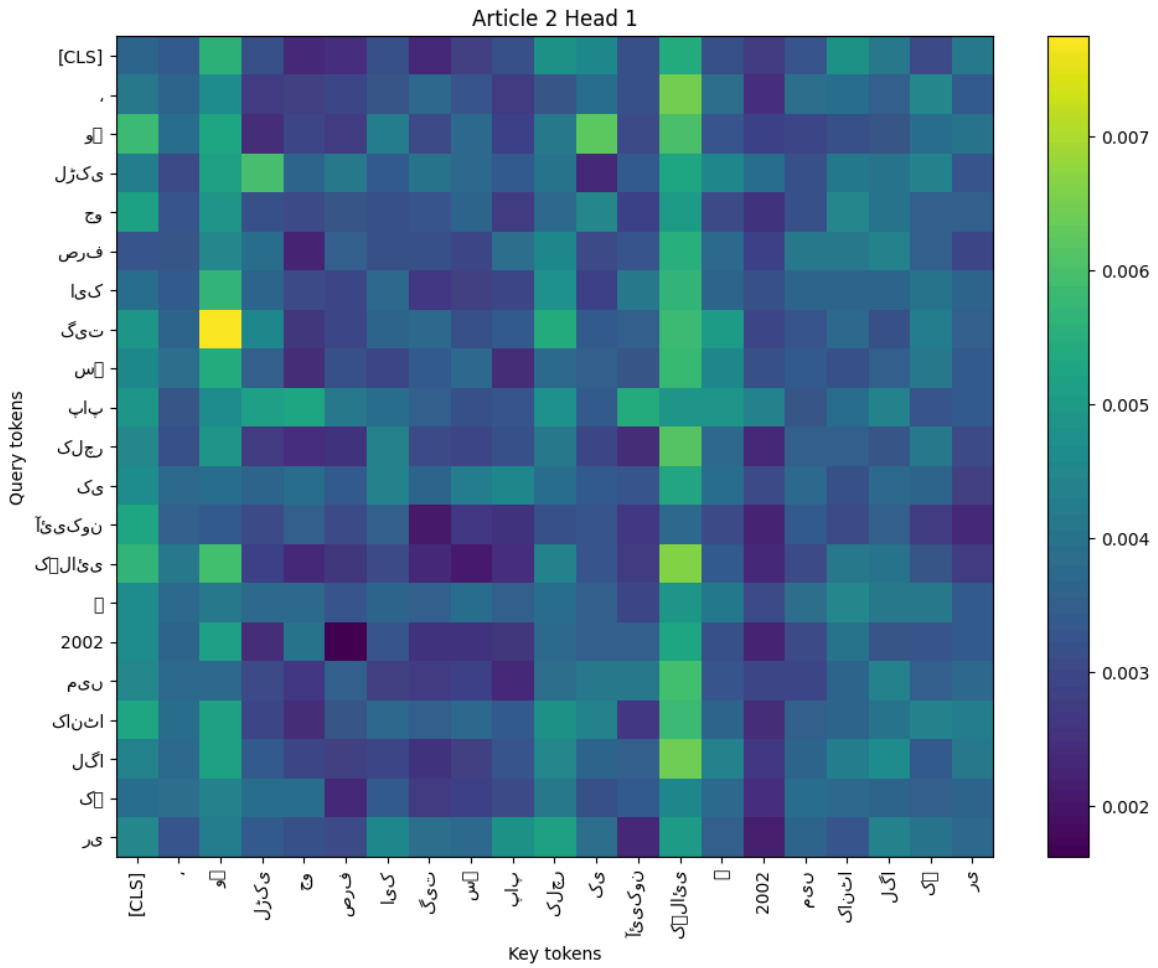
Figure 8: Attention Heatmaps

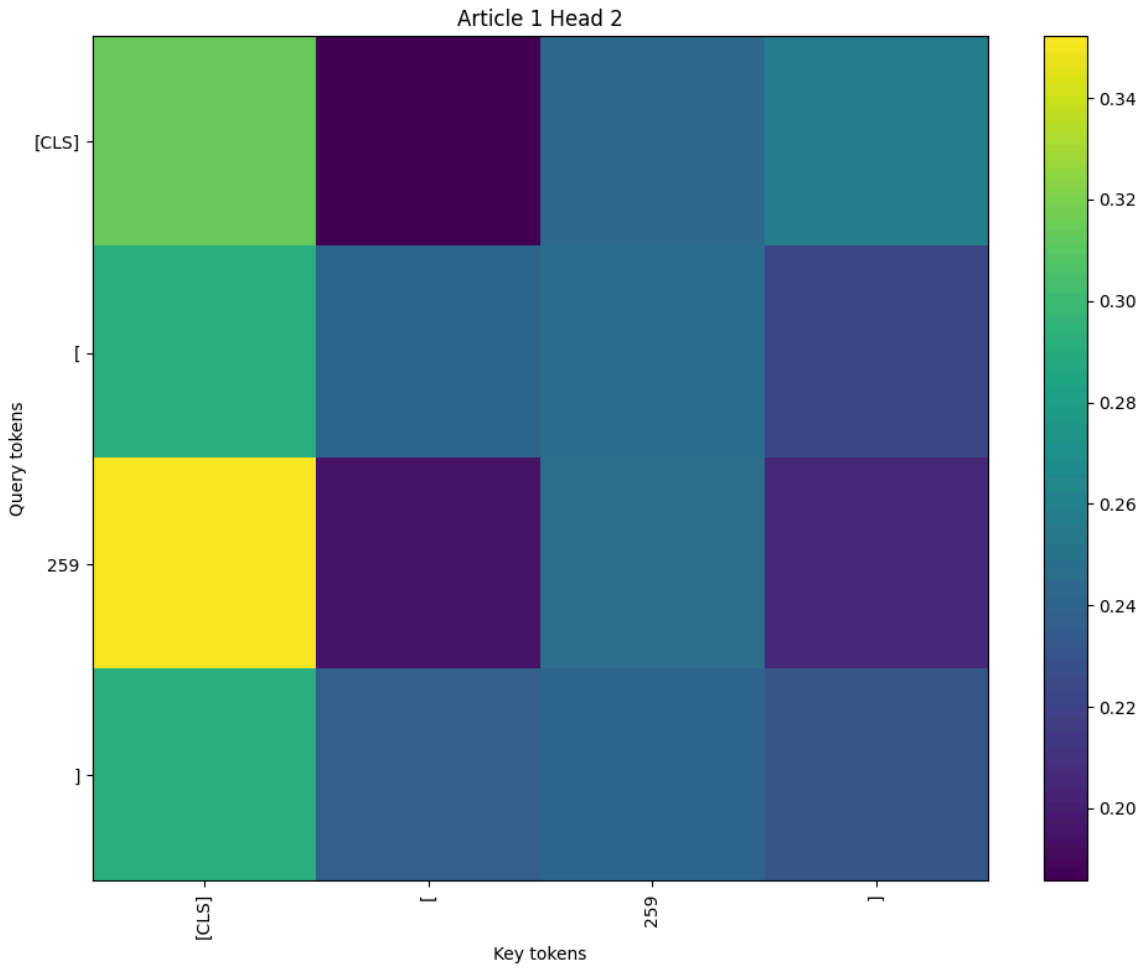

Article 1 Head 1
Query tokens
[CLS]
[
259
]
Key tokens
[CLS]
[
259
]
0.35
0.30
0.25
0.20
0.15


Article 3 Head 2
Query tokens
Key tokens
[CLS]
کسی
ملک
کی
سرحد
پر
پاسپورٹ
افسر
کو
دستاویز
دیا
اور
اس
پر
م□ر
لگوا
،
جو
آپ
کی
آمد
0.007
0.006
0.005
0.004
0.003


Article 3 Head 1
Query tokens
Key tokens
[CLS]
کسی
ملک
کی
سرحد
پر
پاسپورٹ
افسر
کو
دستاویز
دیتا
اور
اس
پر
م□ر
لگواتا
،
جو
آپ
کی
آمد
0.006
0.005
0.004
0.003
0.002


Article 2 Head 2
Query tokens
Key tokens
[CLS]
،
و
لژکی
حق
صرف
اینک
گیت
س
پاپ
کلج
کی
آئیکون
کالائی
2002
مین
کانتا
لگا
ک
ری
0.007
0.006
0.005
0.004
0.003
0.002


Article 2 Head 1
Query tokens
Key tokens
[CLS]
،
2002
0.007
0.006
0.005
0.004
0.003
0.002



4.3 Analysis

The observed performance can be directly attributed to the underlying dataset characteristics. The dataset used for classification contains only a single dominant class (Health & Society), with no meaningful representation of other categories such as politics, sports, or economy.

This results in a trivial learning scenario:

- The model learns that predicting the dominant class yields maximum accuracy
- No penalty is incurred for ignoring other classes
- The model converges quickly to a constant prediction strategy

This phenomenon leads to:

- **Perfect accuracy** (since all labels belong to the same class)
- **Low macro-F1** (since other classes are never predicted)

The confusion matrix further confirms this behavior, showing that all predictions fall into a single class. This is a classic example of how accuracy can be misleading in imbalanced datasets.

From a machine learning perspective, this highlights the importance of:

1. **Balanced datasets**
2. **Appropriate evaluation metrics**
3. **Critical interpretation of results**

The Transformer architecture itself is not inherently flawed. Instead, the issue arises from the absence of a diverse training signal. Transformers rely heavily on large and varied datasets to learn meaningful patterns. In the absence of such data, the model defaults to trivial solutions.

4.4 Attention Analysis

Attention heatmaps provide insight into the internal behavior of the Transformer model. These visualizations illustrate how attention weights are distributed across tokens for different heads.

In this assignment, attention maps show that the model focuses on certain prominent tokens, particularly those that frequently occur across the dataset. The [CLS] token also receives significant attention, as it aggregates information for classification.

However, despite these seemingly meaningful attention patterns, the model fails to perform effective classification. This suggests that:

- Attention mechanisms are functioning correctly at a computational level
- The learned patterns are not discriminative enough for classification

This reinforces the idea that **correct model implementation does not guarantee meaningful learning outcomes**, especially when the dataset lacks diversity.

5. BiLSTM vs Transformer Comparison

The comparison between BiLSTM and Transformer models reveals important insights into the relationship between model architecture and dataset characteristics.

Firstly, in terms of accuracy, the Transformer model appears superior, achieving 100% accuracy. However, this performance is misleading due to dataset imbalance. The BiLSTM model, although achieving lower accuracy, provides more meaningful learning, particularly in the POS tagging task where it demonstrates strong generalization.

Secondly, regarding convergence behavior, the Transformer model converges rapidly due to the simplicity of the classification task. Since all samples belong to a single class, the model quickly learns to output a constant prediction. In contrast, the BiLSTM model requires more epochs to converge, as it learns complex token-level patterns for sequence labeling.

Thirdly, in terms of computational efficiency, Transformers are generally more parallelizable than recurrent models. However, for small datasets, the overhead of attention mechanisms may not provide significant advantages. The BiLSTM model, despite being sequential, is more efficient in capturing local dependencies in such scenarios.

The attention heatmaps generated by the Transformer provide some interpretability, showing which tokens influence predictions. However, due to the lack of class diversity, these insights do not translate into meaningful classification behavior.

Finally, when considering dataset size, the BiLSTM model is more appropriate for small datasets (200–300 samples). Its inductive bias toward sequential data allows it to learn effectively even with limited examples. In contrast, Transformers require large datasets to fully leverage their capacity and learn robust attention patterns.

6. Conclusion

This assignment provides a comprehensive evaluation of multiple NLP techniques and highlights several critical insights.

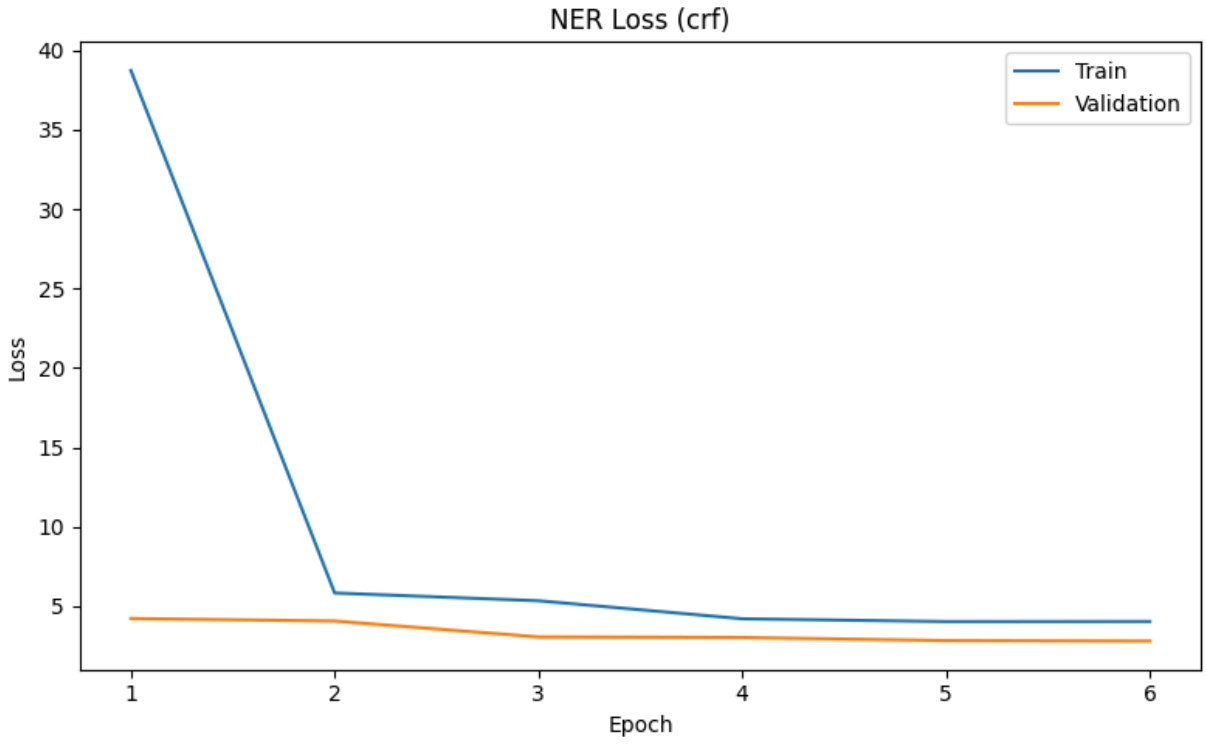
First, the quality and distribution of data play a more significant role than model complexity. Even advanced architectures such as Transformers fail when trained on imbalanced datasets.

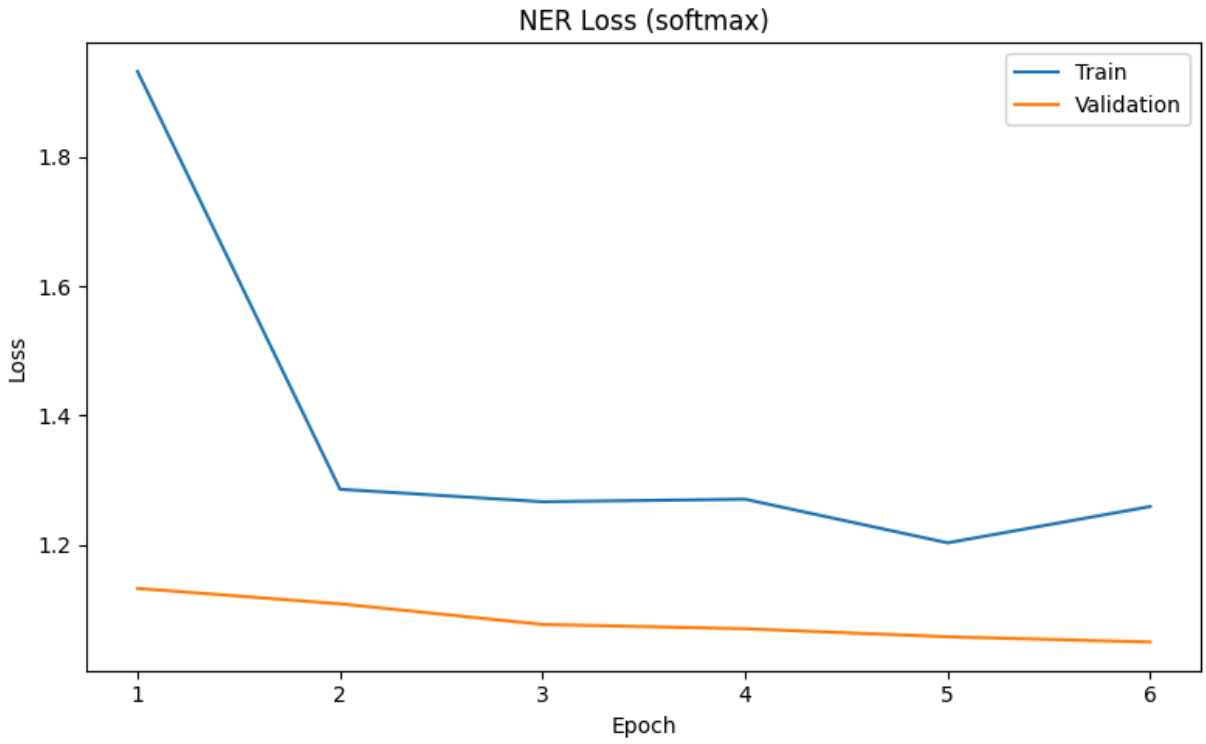
Second, evaluation metrics must be carefully interpreted. Accuracy alone is insufficient, particularly in classification tasks with skewed distributions. Metrics such as macro-F1 provide a more reliable assessment of model performance.

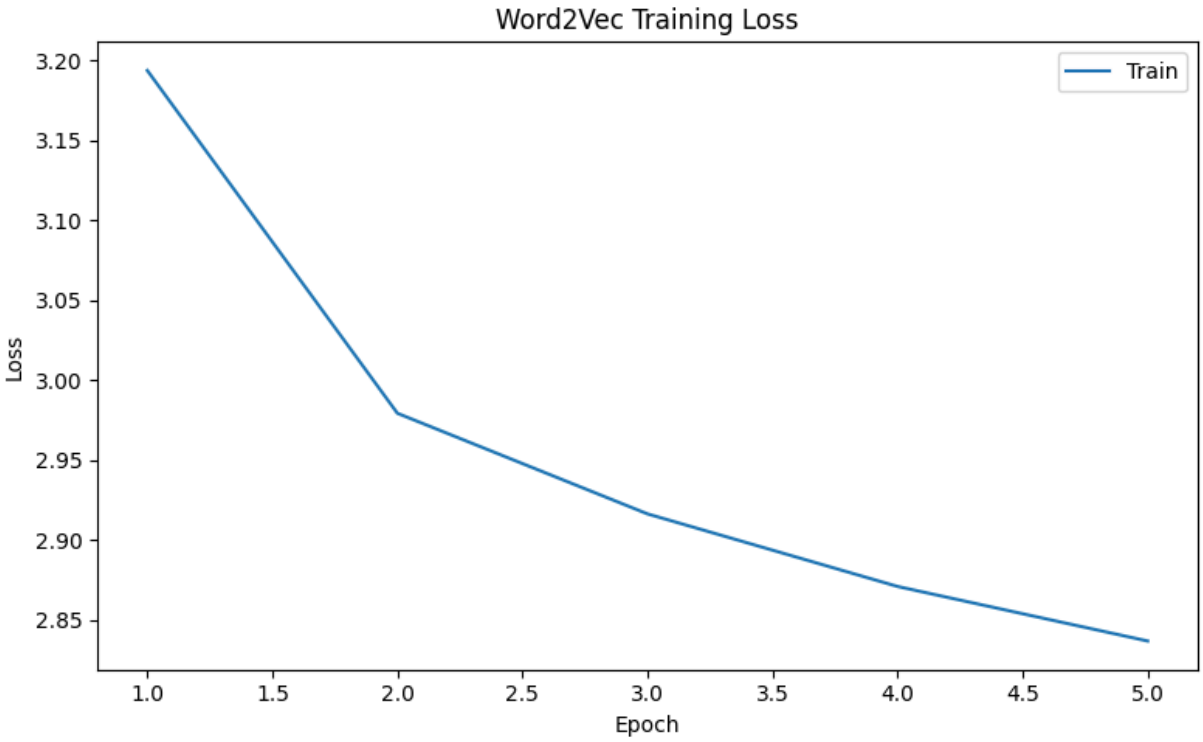
Third, simpler models such as BiLSTM can outperform more complex architectures when data is limited. This underscores the importance of selecting models based on the problem context rather than theoretical superiority.

Finally, the assignment demonstrates that building models from scratch provides deeper understanding of underlying mechanisms, enabling more effective analysis of both successes and failures.

Future work should focus on improving dataset balance, incorporating data augmentation techniques, and exploring hybrid architectures that combine the strengths of recurrent and attention-based models.


NER Loss (crf)
Train
Validation
Loss
Epoch
40
35
30
25
20
15
10
5
1
2
3
4
5
6


NER Loss (softmax)
Train
Validation
Loss
Epoch
1.8
1.6
1.4
1.2
1
2
3
4
5
6




```
PS C:\Users\DELL\Downloads\nlp_assignment2_code (1)\nlp_assignment2> git push origin dev
Uploading LFS objects:  78% (
Uploading LFS objects:  78% (
Uploading LFS objects:  78% (
Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploadin
g LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS ob
jects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (Uploading LFS objects:  78% (7/9), 136 MB | 166 KB/s
Uploading LFS objects:  78% (7/9), 239 MB | 126 KB/s
```